第4章 串

4.1 串类型的定义

4.2 串的表示和实现

4.3 串的模式匹配算法

本章重点难点

重点:

- ADT串的设计、实现方法和基本操作;
- 串的简单模式匹配算法、KMP算法。

难点:

- 串的模式匹配算法中的KMP算法。

第4章 串

- 4.1 串类型的定义
- 4.2 串的表示和实现
- 4.3 串的模式匹配算法

4.1 串类型的定义

□ 串(String)的定义

串是由零个或多个字符组成的有限序列，记为：

$$s=“a_1a_2\ldots a_n” \quad (n\geq 0)$$

其中，s是串的名，用双引号括起来的字符序列是串的值。

□ 串的有关术语

(1) 串的长度: 串中字符的数目n。

(2) 空串(Null string): 长度为零的串。

(3) 子串: 串中任意个连续的字符组成的子序列。

4.1 串类型的定义

□ 串的有关术语

(4) **主串:** 包含子串的串相应地称为主串。

(5) **串相等:** 只有当两个串的长度相等，并且各个对应位置的字符都相等，称两串相等。

(6) **空格串(空白串)(blank string):** 由一个或多个空格组成的串。要和“空串”区别，空格串有长度，就是空格的个数。

4.1 串类型的定义

□ 串的ADT定义

ADT String {

数据对象：$D = \{ a_i \mid a_i \in CharacterSet, i=1,2,\ldots,n, n \geq 0 \}$

数据关系：$R_1 = \{ < a_{i-1}, a_i > \mid a_{i-1}, a_i \in D, i=2,\ldots,n \}$

基本操作：见下页

} ADT String

4.1 串类型的定义

□ 基本操作

StrAssign (&T, chars)	//根据串常量chars生成串T
DestroyString(&S)	//销毁串S
StrCopy (&T, S)	//把串S中内容拷贝到T串
StrLength(S)	//求串长
StrCompare (S, T)	//比较串S和T
Concat (&T, S1, S2)	//连接串
StrEmpty (S)	//判断串是否空

4.1 串类型的定义

□ 基本操作

SubString (&Sub, S, pos, len) //求子串

ClearString (&S) //清空串S

Index (S, T, pos) //子串定位

Replace (&S, T, V) //把串S中符合T的子串替换

StrInsert (&S, pos, T) //插入子串

StrDelete (&S, pos, len) //删除子串

4.2 串的表示和实现

4.2.1、定长顺序存储表示

4.2.2、堆分配存储表示

4.2.3、串的块链存储表示

4.2.1 定长顺序存储表示

- 串的顺序存储C语言实现

```
#define  MAXSTRLEN  255
    // 用户可在255以内定义最大串长
typedef unsigned char  Sstring[MAXSTRLEN+1];
    // 0号单元存放串的长度

Sstring  S;
```

4.2 串的表示和实现

4.2.1、定长顺序存储表示

4.2.2、堆分配存储表示

4.2.3、串的块链存储表示

4.2.2 堆分配存储表示

- 堆分配存储表示的C语言实现

```
typedef struct {
  char *ch;
        // 若是非空串，则按串长分配存储区，
        //  否则ch为NULL
  int  length;
        // 串长度
} HString;
```

4.2 串的表示和实现

4.2.1、定长顺序存储表示

4.2.2、堆分配存储表示

4.2.3、串的块链存储表示

4.2.3 串的块链存储表示

□ 链表存储字符串的讨论

字符串本身就是一个线性表，可以用链表存储。

如果每个结点存储一个字符，如采用32位地址，字符按8位记，则存储密度是多少？

$$\text{存储密度} = \frac{\text{数据元素所占存储位}}{\text{实际分配的存储位}}$$

$$\text{存储密度} = \frac{8}{40} = 20\%$$

4.2.3 串的块链存储表示

- 链表存储字符串的讨论

结论：采用普通链表存储字符串，存储密度非常低，浪费空间严重。

解决办法：一个结点存储多个字符。这就是串的块链存储。

4.2.3 串的块链存储表示

- 串的块链存储的C语言实现

```
#define  CHUNKSIZE  80
typedef  struct Chunk {  // 结点结构
  char  ch[CUNKSIZE];
  struct Chunk  *next;
} Chunk;
typedef struct {          // 串的链表结构
  Chunk *head, *tail; // 串的头和尾指针
   int   curlen;          // 串的当前长度
} LString;
```

4.3 串的模式匹配算法

4.3.1 模式匹配简单算法

4.3.2 模式匹配KMP算法

4.3.1 模式匹配简单算法（Brute-Force）

设S为目标串，T为模式串，且不妨设：

$S=“s_0s_1s_2s_3\ldots\ldots s_{n-1}”$， $T=“t_0t_1t_2t_3\ldots t_{m-1}”$

串的匹配实际上是对合法的位置$0\leqslant i\leqslant n-m$依次将目标串中的子串s[i…i+m-1]和模式串t[0…m-1]进行比较：

- 若s[i…i+m-1]=t[0…m-1]：则称从位置i开始的匹配成功，亦称模式t在目标s中出现；
- 位置i称为位移，当s[i…i+m-1]=t[0…m-1]时，i称为有效位移；

串匹配问题可描述为找出某给定模式T在目标串S中首次出现的有效位移。

4.3.1 模式匹配简单算法（Brute-Force）

$S=“s_0s_1s_2s_3……s_n”$

$T=“t_0t_1t_2t_3 ……t_m”$

```
int Index(Sstring S, Sstring T, int pos) {
    i = pos;   j = 1;//首次对齐位置
   while (i <= S[0] && j <= T[0]) {
    if (S[i] == T[j]) { ++i;  ++j; }   // 继续比较后继字符
    else
    { i = i-j+2;   j = 1; }            // 指针后退重新开始匹配
   }
  if (j > T[0])  return  i-T[0];
  else return 0;
} // Index
```

```
#define  MAXSTRLEN  255
typedef unsigned char
Sstring[MAXSTRLEN+1];
```

4.3.1 模式匹配简单算法

□ 时间复杂性分析

讨论下面这种情况的时间复杂性，设S串长为n，T串长为m。

S：a b c d e f g h j k l l k c d e　　长为n

T：j k l　　长为m

✓ 若第i趟比较成功，共比较了多少次？

- 假设从第i个位置匹配成功
  - 前i-1个位置共比较了 i-1 次
  - 第i个位置开始比较了 m 次
  - 共比较了i+m-1次。
- i从1到 n-m+1 ，共(n+m)(n-m+1)/2
- 平均需比较(n+m)/2
- 最好的情况平均时间复杂度为O(n+m)

4.3.1 模式匹配简单算法

◻ 时间复杂性分析

讨论下面这种情况的时间复杂性，设S串长为n，T串长为m。

S：0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1

T：0 0 0 0 0 0 1

- 若第i趟比较成功，共比较了多少次？
  - ✓ 前i-1趟比较每次都比较m 次，第i趟也比较 m 次。
  - ✓ 共i*m次。
  - ✓ i从1到n-m+1，共比较了(n-m+2)(n-m+1)m/2
  - ✓ 平均比较次数(n-m+2)m/2
  - ✓ 最坏的情况时间复杂度为O(n×m)

4.3 串的模式匹配算法

4.3.1 模式匹配简单算法

4.3.2 模式匹配KMP算法

- 该改进算法是由D.E.Knuth，J.H.Morris和V.R.Pratt提出来的，简称为KMP算法。

4.3.1 模式匹配KMP算法

□ 事例讨论

例

下图中主串游标i指向8，子串中游标j指向8，按照简单模式匹配算法两处不相等时j回到1，i回到2，继续比较，分析在这种情况下，这样做有没有意义？怎么做才有意义？

	1	2	3	4	5	6	7	8	9				
主串S:	a	b	c	d	a	b	c	d	a	b	c	f	e
								i					
子串T:	a	b	c	d	a	b	c	f					
								j					

结论：j回到1，i回到2没有意义。

4.3.1 模式匹配KMP算法

□ 事例讨论

结论：可以i不动，j回到4。如下图。

	1	2	3	4	5	6	7	8	9				
主串S:	a	b	c	d	a	b	c	d	a	b	c	f	e
								i					
子串T:	a	b	c	d	a	b	c	f					
				j									

□ KMP算法的思想

主串指针不回溯，模式串向后滑动至某个位置上。

4.3.1 模式匹配KMP算法

□ KMP算法游标定位的几种情况

(1) 第一个字符不相等。

	1	2	3	4	5	6	7	8	9				
主串S:	a	b	c	d	a	b	c	d	a	b	c	f	e
	i												
	j												
子串T:	b	b	c	d	a	b	c	f					

此时：
i++;

4.3.1 模式匹配KMP算法

□ KMP算法游标定位的几种情况

(2) 在子串j处不相等，在子串中j之前没有“前缀和后缀相等的情况”。

	1	2	3	4	5	6	7	8	9				
主串S:	a	b	c	d	a	b	c	d	a	b	c	f	e
					i								
					j								
子串T:	a	b	c	d	b	b	c	f					

此时：
i不动，j回到1

4.3.1 模式匹配KMP算法

□ KMP算法游标定位的几种情况

(3) 在子串j处不相等，在子串中j之前存在“前缀和后缀相等的情况”。

主串S:

1	2	3	4	5	6	7	8	9				
a	b	c	d	a	b	c	d	a	b	c	f	e
							i					

子串T:

a	b	c	d	a	b	c	f
							j

此时：回到“最长前后缀相等串”+1的位置

4.3.1 模式匹配KMP算法

□ KMP算法游标定位的几种情况

j游标回到什么位置（一般称作next值），有三种情况：

(1) 第一个字符不相等。

此时：i++; 约定next值为0

(2) 在子串j处不相等，在子串中j之前没有“前缀和后缀相等的情况”。

此时：i不动，j回到1；next值为1

(3) 在子串j处不相等，在子串中j之前存在“前缀和后缀相等的情况”。

此时： i不动，回到“最长前后缀相等串”+1的位置；next值为最长前后相等子串的长度+1；

4.3.1 模式匹配KMP算法

□ KMP算法

j游标回到什么位置（一般称作next值），有三种情况：

(1) 第一个字符不相等。	此时：i++; 约定next值为0

(2) 在子串j处不相等，在子串中j之前没有“前缀和后缀相等的情况”。

此时：i不动，j回到1；next值为1

(3) 在子串j处不相等，在子串中j之前存在“前缀和后缀相等的情况”。

此时：i不动，回到“最长前后缀相等串”+1的位置；next值为最长前后相等子串的长度+1；

```
int Index_KMP (SString S,SS
{      i= pos,  j =1;
       while (i<=S[0] && j<=T[0])
          if (j==0 || S[i]==T[j])  { i++;j++; }
          else
             j=next[j];                    //i不变,j后退

       if (j>t[0])  return i-t[0];  //匹配成功
       else   return 0;                    //返回不匹配标志
}
```

模式匹配KMP算法

□ next[j]函数

表明当模式中第j个字符与主串中相应字符“失配”时，在模式中需重新和主串中该字符进行比较的字符的位置。

$$
\text{next}[j]=\begin{cases}\max\{\, k \mid 1<k<j,\ \text{且“}t_1\ldots t_{k-1}\text{”} == \text{“}t_{j-k+1}\ldots t_{j-1}\text{”}\,\} & \text{当此集合非空时} \\ 0 & \text{当}j=1\text{时} \\ 1 & \text{其他情况}\end{cases}
$$

- **没有回溯的模式匹配算法：用 S[i] 与 T[j] 匹配，若相等则往后继续匹配，否则就用S[i]与T[next[j]]匹配，直至匹配成功或失败为止**

4.3.1 模式匹配KMP算法

□ 例题

给出下面字符串各位的next值：

	1	2	3	4	5	6	7	8	9
T	a	b	a	b	a	b	a	b	e
next值	0	1	1	2	3	4	5	6	7

$$
\text{next}[j]=\begin{cases}\max\{\, k \mid 1<k<j,\ \text{且}\ \text{“}t_1\ldots t_{k-1}\text{”} == \text{“}t_{j-k+1}\ldots t_{j-1}\text{”}\,\} & \text{当此集合非空时} \\ 0 & \text{当}j=1\text{时} \\ 1 & \text{其他情况}\end{cases}
$$

单选题 1分

设置

KMP算法的过程中，若某次比对在模式串P的第j个位置P[j]处失败，则将对齐位置换为：

prev[j]

next[j]

prev[j] + 1

next[j] + 1

单选题 1分

设置

例题

给出字符串abaabab的next值(未经优化，下标从1开始计数)

0112234

0001123

0112233

0223345

4.3.1 模式匹配KMP算法

□ 求next函数值

(1) next[1] = 0;表明主串从下一字符s_{i+1}起和模式串重新开始匹配。i = i+1; j = 1;

(2) 设next[j] = k，（1<k<j）则next[j+1] = ?

① 若$t_k = t_j$，则有“$t_1 \ldots t_{k-1} t_k$” = “$t_{j-k+1} \ldots t_{j-1} t_j$”，如果在j+1发生不匹配:

说明next[j+1] = k+1 = next[j]+1。

② 若$t_k \neq t_j$，可把求next值问题看成是一个模式匹配问题，整个模式串既是主串，又是子串。

4.3.1 模式匹配KMP算法

□ 求next函数值

设next[j] = k,则有“$t_1 \ldots t_{k-1}$” = “$t_{j-k+1} \ldots t_{j-1}$”。求next[j+1]

- 如果t[j]=t[k], “$t_1 \ldots t_{k-1} t_k$” = “$t_{j-k+1} \ldots t_{j-1} t_j$”则 next[j+1]=k+1;
- 如果t[j] ≠t[k]

若$t_{k'}=t_j$，则有“$t_1 \ldots t_{k'}$”=“$t_{j-k'+1} \ldots t_j$”,

next[j+1]=k’+1=next[k]+1=next[next[j]]+1.

若$t_{k''}=t_j$，则有“$t_1 \ldots t_{k''}$”=“$t_{j-k''+1} \ldots t_j$”,

next[j+1]=k”+1=next[k’]+1=next[next[k]]+1.

next[j+1]=1.

4.3.1 模式匹配KMP算法

1	2	3	4	5	6			
a	b	a	b	e	b			
0	1	1	2	3	1			

□ 求next函数值

设next[j] = k,求next[j+1]

- 如果t[j]=t[k], “$t_1 \ldots t_{k-1} t_k$
  next[j+1]=k+1;
- 如果t[j] ≠t[k]

若$t_{k'}=t_j$，则有“$t_1 \ldots t_{k'}$”=“t

next[j+1]=k’+1=next[k]+

若$t_{k''}=t_j$，则有“$t_1 \ldots t_{k''}$”=“

next[j+1]=k”+1=next[k’]

next[j+1]=1.

求next[6]过程：

前序：ab == ab next[5]=k=3

aba == abe? No

Need next[3]=1

a == e ? No

Need Next[1] = 0

Next[6]=1

4.3.1 模式匹配KMP算法

□ 求next函数算法

1	2	3	4	5	6			
a	b	a	b	e	b			
0	1	1	2	3	1			

```
void get_next(SString T,int next[])
{  int i=1,    j=0;  next[1]=0;
   while(i<T[0]){
     if(j==0||T[i]==T[j])
        {++i; ++j; next[i]=j;}
    else     j=next[j] ;
   }
}
```

j=0 i=1 next[1]=0

j=0 → j=1 i=2 next[2]=1

T[2]<>T[1]) → j=next[j]=0

j=0 → j=1 i=3 next[3]=1

T[3]==T[1]) → j=2 i=4 next[4]=2

T[4]==T[2]) → j=3 i=5 next[5]=3

T[5]<>T[3]) → j=next[3]=1

T[5]<>T[1]) → j=next[1]=0

单选题 1分

设置

next[j]之后下一个尝试的位置是:

next[j] - 1

next[j] + 1

next[j] * 2

next[next[j]]

本章小结

熟练掌握：

(1)串的定义、性质和特点；

(2)ADT串的设计、实现方法和基本操作；

(3)朴素模式匹配算法；